

MAKERS E DESENVOLVEDORES

Livro do Aluno • 13 a 17 anos

Do raciocínio algorítmico a projetos reais e publicados

Material gratuito e aberto para uso educacional

Baseado em **Computer Science Unplugged** e nas documentações oficiais de Python, SQLite, Git e GitHub

github.com/Marquezbertin

SUMÁRIO

- 1. Como pensar como um programador
- 2. Algoritmos e estruturas de controle
- 3. Python: a linguagem da primeira programação
- 4. Desenvolvimento web: HTML, CSS e JavaScript
- 5. Banco de dados e SQL
- 6. Git e GitHub: trabalho em equipe
- 7. Qualidade de software e testes
- 8. Inteligência artificial: fundamentos e uso responsável
- 9. Metodologias ágeis, cibersegurança e ética
- 10. Projeto final: tecnologia a serviço da comunidade
- 11. Inglês técnico para programadores
- 12. Recapitulando e continuando
- 13. Planner do projeto final
- 14. Links rápidos de ferramentas
- 15. Glossário

CAPÍTULO 1 - COMO PENSAR COM UM PROGRAMADOR

> **Objetivo:** Dominar as quatro habilidades do pensamento computacional e aplicá-las em problemas reais.

Pensamento computacional não é uma matéria exótica. É o método que as pessoas usam para resolver problemas de forma sistemática: entender o problema, quebrá-lo em partes, identificar o que se repete, decidir o que realmente importa e, por fim, descrever a solução como uma sequência precisa de passos. Depois que você domina esse método, a linguagem de programação passa a ser apenas a ferramenta para escrever a solução no computador.

1.1 Os quatro pilares

Decomposição. Um problema grande é dividido em subproblemas menores e gerenciáveis. Um sistema de biblioteca, por exemplo, se divide em catálogo, busca, empréstimo e relatórios. Cada parte pode ser resolvida e testada separadamente.

Reconhecimento de padrões. Problemas parecidos têm soluções parecidas. Reconhecer o padrão que se repete é o que permite escrever o código uma vez e reutilizá-lo em vários lugares, em vez de copiar e colar.

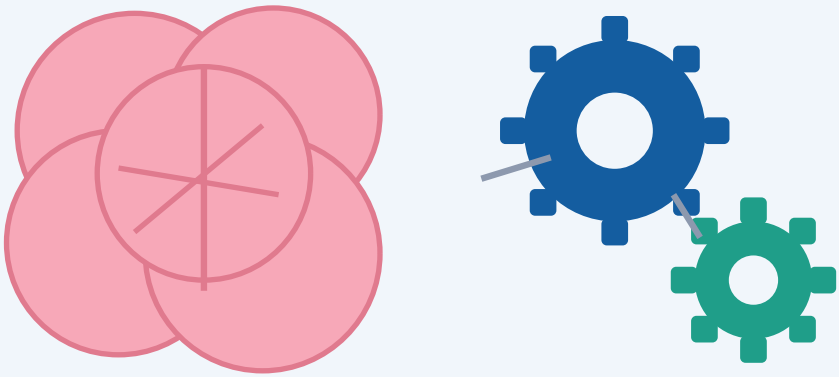
Abstração. Separar o essencial do acidental. Ao trabalhar com um banco de dados, você não precisa saber o algoritmo de busca de disco: precisa saber que existe uma função que faz a busca. A abstração é a economia de trabalho mental que permite projetos grandes.

Algoritmos. Um algoritmo é uma sequência finita e não ambígua de passos que resolve um problema. Para ser considerado bom, um algoritmo deve ter **correção** (faz o que promete), **eficiência** (faz em tempo razoável) e **clareza** (outra pessoa consegue ler e seguir).

1.2 Complexidade: quanto tempo o algoritmo leva

Nem todo algoritmo que funciona é eficiente. Suponha que você procura um nome em uma lista de mil registros. A busca linear percorre a lista do início ao fim: no pior caso, mil comparações. A busca binária, se a lista estiver ordenada, descarta metade a cada passo: com mil registros, são cerca de dez comparações. A diferença cresce de forma dramática conforme o volume cresce.

Essa distinção recebe o nome de **notação big-O**. Ela descreve como o tempo de execução cresce em relação ao tamanho da entrada. **O(n)** significa que o custo dobra quando o volume dobra. **O(n log n)** significa que o custo cresce um pouco mais que o dobro, mas continua muito melhor do que O(n). **O(log n)** significa que o custo cresce de forma logarítmica: mesmo com um bilhão de itens, bastam cerca de trinta passos.



Os quatro pilares do pensamento computacional

Mão na massa - Atividade 1

Implemente, em pseudocódigo, a busca linear e a busca binária. Depois escreva um programa que gere uma lista de mil números aleatórios, embaralhe-a, ordene-a e compare o número de comparações das duas abordagens. Documente os resultados numa tabela.

CAPÍTULO 2 - ALGORITMOS E ESTRUTURAS DE CONTROLE

> **Objetivo:** Escrever algoritmos corretos, com estruturas de decisão, repetição, funções e recursão.

Toda linguagem de programação oferece as mesmas estruturas fundamentais. **Sequência** é a forma mais básica: uma instrução depois da outra. **Decisão** (if/elif/else) permite que o caminho do algoritmo dependa de uma condição. **Repetição** (for/while) permite executar um bloco várias vezes até que uma condição deixe de ser verdadeira.

2.1 As três estruturas que resolvem quase tudo

```
SE condicao:
    faca_A
SENAO:
    faca_B

ENQUANTO condicao:
    faca_C

PARA cada_item DE sequencia:
    faca_D
```

O **if** é a estrutura de decisão, o **while** é a repetição controlada por uma condição e o **for** é a repetição controlada por um contador ou por uma sequência. Quase todo algoritmo pode ser escrito com essas três estruturas, mais chamadas de função e mais variáveis. Esse é um resultado fundamental: é o que torna a computação tão versátil.

2.2 Funções: nomear e reaproveitar

Uma função é um bloco de código com nome, que recebe parâmetros, executa uma tarefa e devolve um resultado. O ganho não é apenas evitar repetição: é que cada função pode ser testada isoladamente, e é isso que permite escrever software confiável.

```
def media(notas):
    """Retorna a media aritmetica de uma lista de notas."""
    if len(notas) == 0:
        raise ValueError("lista vazia")
    return sum(notas) / len(notas)

def classificacao(m):
    if m >= 9: return "A"
    elif m >= 7: return "B"
    elif m >= 5: return "C"
    return "D"
```

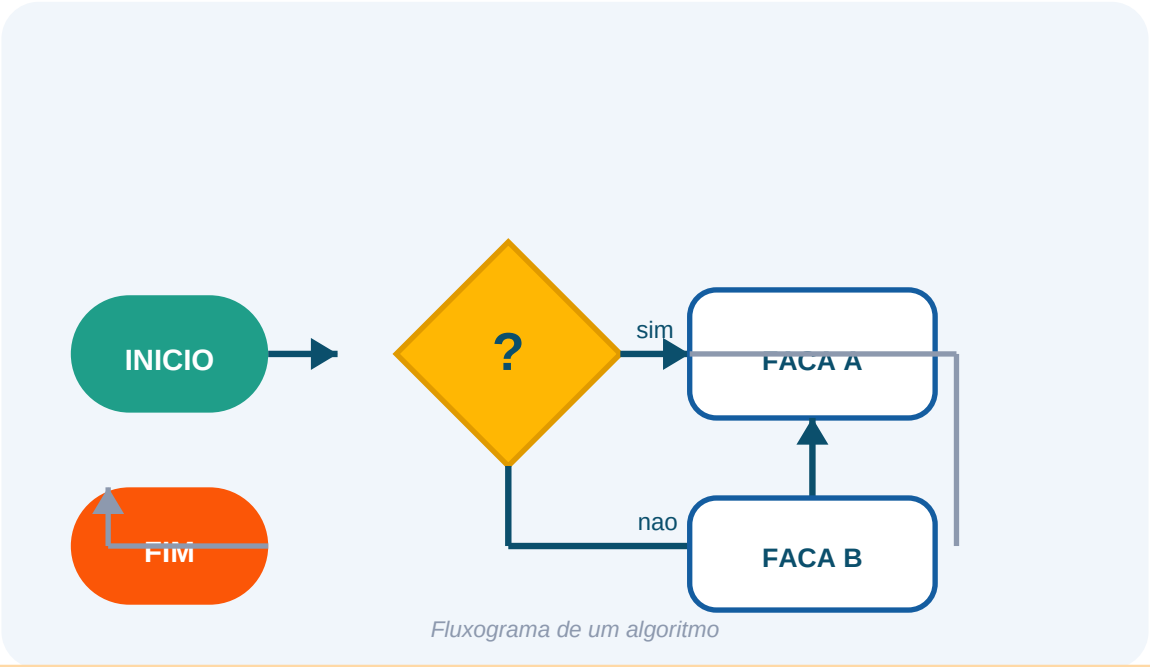
Repare no **docstring**, que é a documentação da função escrita logo abaixo do **def**. E no **raise ValueError**, que transforma um caso inválido em um erro explícito. Um bom programador não espera que o usuário seja sempre razoável: ele trata o erro antes que ele aconteça.

2.3 Recursão: a função que se chama

Uma função recursiva resolve um problema chamando a si mesma com um caso menor, até atingir um **caso base** que encerra a recursão. A definição clássica é a de fatorial, em que o fatorial de *n* é *n* multiplicado pelo fatorial de *n* menos 1.

```
def fatorial(n):
    if n == 0:      # caso base: encerra a recursao
        return 1
    return n * fatorial(n - 1)
```

Sem o caso base, a função chama a si mesma indefinidamente e o programa estoura a pilha de chamadas, resultando em **RecursionError**. A recursão é elegante, mas nem sempre é a melhor escolha: em Python, um laço comum costuma ser mais rápido que a versão recursiva equivalente, porque não cria novos quadros de pilha.



Fluxograma de um algoritmo

Desafio

Implemente uma função recursiva que calcule a soma de todos os números de 1 até *n*. Depois reescreva com laço e compare o tempo de execução para *n* igual a 1.000, 10.000 e 100.000.

CAPÍTULO 3 - PYTHON: A LINGUAGEM DA PRIMEIRA PROGRAMAÇÃO

> **Objetivo:** Escrever programas Python legíveis, robustos e testáveis.

Python foi criado por Guido van Rossum e publicado em 1991. A proposta central era a legibilidade: o código deveria se parecer com a forma como um humano escreve a mesma ideia. Por isso Python é amplamente usado em ensino, ciência de dados, automação e desenvolvimento web.

3.1 Os tipos fundamentais

```
idade = 16              # int - numero inteiro
altura = 1.72           # float - numero real
nome = "Ana"            # str - texto
estudante = True        # bool - verdadeiro ou falso
notas = [8.5, 9.0, 7.0] # list - sequencia mutavel
perfil = {"nome": "Ana", "idade": 16}  # dict - pares chave:valor
```

Python possui tipagem dinâmica: o tipo é determinado pelo valor atribuído, e não declarado. Isso torna o código curto, mas exige atenção. A operação **idade + 1** funciona quando *idade* é inteiro e falha quando *idade* é texto, porque a função **input()** sempre devolve uma string.

3.2 Entrada, saída e f-strings

```
nome = input("Qual o seu nome? ").strip()
nota = float(input("Qual a sua nota? "))
resultado = "aprovou" if nota >= 7 else "reprovou"
print(f"Ola, {nome}! Voce tirou {nota:.1f} e {resultado}.")
```

A **f-string**, escrita com o prefixo **f** antes das aspas, é o jeito mais limpo de juntar texto e valores. O especificador **:.1f** arredonda para uma casa decimal. A expressão condicional dentro das chaves permite decidir o texto sem um **if** separado.

3.3 Tratamento de erros

Programas quebram. A diferença entre um programa amador e um profissional está em como ele reage ao imprevisto. O bloco **try/except** captura erros previsíveis e permite que o programa se recupere ou ofereça uma mensagem útil ao usuário.

```
while True:
    try:
        n = int(input("Digite um numero: "))
        break
    except ValueError:
        print("Isso nao e um numero. Tente novamente.")
```

Esse padrão, chamado de **validação por repetição**, é uma das marcas de um programa bem feito. Ele transforma um erro em um caminho previsto e ensinado ao usuário.



Mão na massa - Atividade 2

Escreva um programa que leia nome, data de nascimento e três notas, calcule a média com f-string, classifique o resultado e salve todos os registros em uma lista de dicionários. Valide cada entrada com try/except e recuse datas impossíveis. Escreva pelo menos três testes automatizados com **assert**.

CAPÍTULO 4 - DESENVOLVIMENTO WEB: HTML, CSS E JAVASCRIPT

> **Objetivo:** Construir páginas web semânticas, responsivas e interativas.

A web tem três linguagens fundamentais com papéis bem definidos. **HTML** define a estrutura e o significado do conteúdo: um título é um título, um formulário é um formulário. **CSS** define a apresentação: cores, tipografia, espaçamento e layout. **JavaScript** define o comportamento: o que acontece quando o usuário interage.

4.1 HTML semântico

```
   cabecalho da pagina
      navegacao principal
     conteudo principal da pagina
   um bloco tematico
   um conteudo autonomo
    conteudo lateral
   rodape
```

Usar tags semânticas em vez de **div** para tudo tem três vantagens concretas: leitores de tela navegam melhor, buscadores entendem a hierarquia e o código fica legível para quem vier depois. Acessibilidade não é um extra: é uma das razões pelas quais a semântica existe.

4.2 CSS: o modelo de caixa e o layout

Todo elemento em CSS ocupa uma caixa com quatro partes: **conteúdo**, **padding** (espaço interno), **borda** e **margem** (espaço externo). Entender esse modelo resolve a maioria dos problemas de "por que meu layout quebrou".

```
.container {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 1rem;
}

.card {
  padding: 1rem;
  border-radius: 12px;
  background: #f7f9fc;
}
```

Flexbox organiza elementos em uma dimensão, em linha ou em coluna. **Grid** organiza em duas dimensões, linhas e colunas ao mesmo tempo. A propriedade **gap** substitui margens manuais e evita problemas de espaçamento colapsado. A tag **viewport** é obrigatória no **head** para que o site se adapte ao celular.

4.3 JavaScript e o DOM

```
const campo = document.querySelector("#email");
const aviso = document.querySelector("#aviso");
```

```
form.addEventListener("submit", (evento) => {
  evento.preventDefault();
  if (!campo.value.includes("@")) {
    aviso.textContent = "E-mail invalido";
    return;
  }
  aviso.textContent = "Enviado com sucesso!";
});
```

O **DOM** é a representação em objetos do HTML que o navegador constrói. Com **querySelector** você localiza elementos; com **addEventListener** você reage a eventos; com **preventDefault** você impede o comportamento padrão, como recarregar a página no envio de um formulário.



Como a web conecta as páginas

Desafio

Construa uma página de cadastro com validação em JavaScript, retorno imediato para cada campo, layout responsivo com Grid e um contador de caracteres restantes no campo de biografia. Teste nas resoluções de 360 px, 768 px e 1440 px.

CAPÍTULO 5 - BANCO DE DADOS E SQL

> **Objetivo:** Modelar dados relacionais e escrever consultas eficientes em SQL.

Um banco de dados relacional guarda informação em **tabelas**, onde cada linha é um registro e cada coluna é um atributo. A regra de ouro do modelo relacional é simples de enunciar e difícil de respeitar: **nunca repita o mesmo dado em duas linhas**. Se o nome de um aluno aparece em cem notas, guarde o aluno uma vez e relacione as notas a ele por um identificador.

5.1 Modelo: entidade, atributo e relacionamento

A modelagem começa identificando as **entidades** (as coisas do mundo real que queremos guardar: aluno, turma, pedido), seus **atributos** (as características) e os **relacionamentos** com sua cardinalidade. Um relacionamento **um para muitos** significa que um registro se liga a vários da outra tabela; **muitos para muitos** exige uma tabela intermediária.

```
CREATE TABLE alunos (
  id_aluno INTEGER PRIMARY KEY AUTOINCREMENT,
  nome TEXT NOT NULL,
  email TEXT UNIQUE
);

CREATE TABLE matriculas (
  id_matricula INTEGER PRIMARY KEY AUTOINCREMENT,
  id_aluno INTEGER NOT NULL REFERENCES alunos(id_aluno),
  id_turma  INTEGER NOT NULL REFERENCES turmas(id_turma),
  data_matricula DATE NOT NULL
);
```

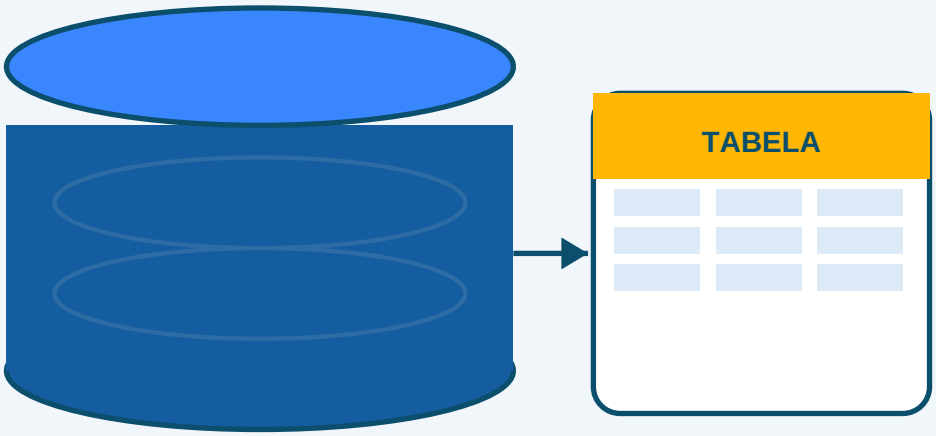
A **chave primária** identifica unicamente cada linha. A **chave estrangeira** cria o vínculo e garante a integridade referencial: o banco recusa uma matrícula para um aluno que não existe. **NOT NULL** proíbe valor ausente, **UNIQUE** impede duplicidade e **AUTOINCREMENT** gera o identificador sequencialmente.

5.2 Consultar: as cinco perguntas

Praticamente toda consulta se resolve com cinco perguntas: **o quê** (quais colunas), **de onde** (quais tabelas), **como filtrar** (**WHERE**), **como agrupar** (**GROUP BY**) e **como ordenar** (**ORDER BY**). A ordem dessas cláusulas na escrita é fixa.

```
SELECT a.nome, AVG(n.valor) AS media
FROM alunos a
JOIN matriculas m ON m.id_aluno = a.id_aluno
JOIN notas n     ON n.id_matricula = m.id_matricula
WHERE a.turma = '1A'
GROUP BY a.id_aluno, a.nome
HAVING AVG(n.valor) >= 7
ORDER BY media DESC
LIMIT 10;
```

A diferença entre **WHERE** e **HAVING** é uma das confusões mais comuns: **WHERE** filtra *linhas* antes do agrupamento, e **HAVING** filtra *grupos* depois. Por isso o filtro por média só pode usar HAVING, já que a média só existe depois do GROUP BY.



Banco de dados: informação organizada e relacionada

Mão na massa - Atividade 3

Modele e implemente o banco de uma biblioteca escolar, com livros, alunos e empréstimos. Escreva as dez consultas mais úteis: quais livros estão disponíveis, quais estão atrasados, o livro mais emprestado, a média de dias de empréstimo por categoria e um relatório mensal. Depois conecte com Python usando **sqlite3**, sempre com parâmetros vinculados, nunca por concatenação de texto.

CAPÍTULO 6 - GIT E GITHUB: TRABALHO EM EQUIPE

> **Objetivo:** Versionar projetos, colaborar por branches e pull requests, e publicar resultados.

Git é um sistema de controle de versão: ele guarda o histórico completo de um projeto, permitindo voltar a qualquer ponto anterior e ver exatamente o que mudou, quando e por quê. GitHub é a plataforma web que hospeda repositórios Git e oferece ferramentas de colaboração em torno deles.

6.1 O ciclo básico

```
git init                      # cria o repositorio local
git clone               # copia um repositorio remoto
git status                    # o que mudou, o que esta pronto
git add .                    # marca as mudancas para o proximo commit
git commit -m "mensagem"     # grava um ponto no historico
git push                     # envia os commits para o remoto
```

O fluxo de trabalho em três áreas é o coração do Git: a **área de trabalho** (onde você edita os arquivos), a **área de preparação** (**git add**) e o **repositório** (**git commit**). O **git status** é o comando que sempre responde o que está acontecendo.

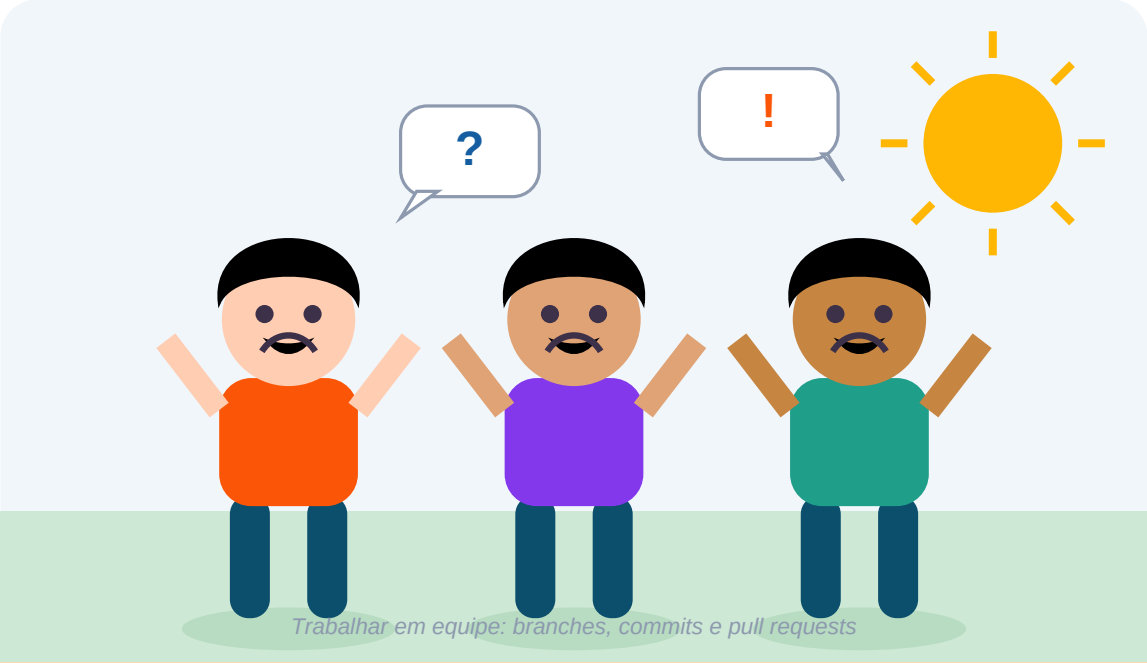
6.2 Branches e merge

```
git checkout -b feature/nova-tela   # cria e entra na branch
git add .
git commit -m "cria nova tela"
git checkout main
git merge feature/nova-tela
```

Uma branch é uma linha de desenvolvimento independente. O fluxo recomendado em equipe é trabalhar em branches curtas, uma pessoa por branch, e integrar por **pull request**, onde o código é revisado antes de entrar na branch principal. Isso não é burocracia: é o que impede que o projeto quebre de forma inesperada.

6.3 Pull request e issues

Um **pull request** é uma proposta de integrar uma branch. Ele permite revisão, comentários linha a linha e verificação automática de testes antes da aprovação. Uma **issue** registra uma tarefa, um defeito ou uma ideia. Juntos, eles são a estrutura mínima de qualquer projeto aberto.



Desafio

Crie um repositório para o seu projeto, faça pelo menos dez commits com mensagens claras, publique-o no GitHub Pages e escreva um README com objetivo, instruções de execução, tecnologias usadas, licença e contribuições. Em seguida, abra um pull request em um repositório aberto de verdade, mesmo que seja apenas a correção de um erro de digitação na documentação.

CAPÍTULO 7 - QUALIDADE DE SOFTWARE E TESTES

> **Objetivo:** Escrever testes automatizados e aplicar os principais tipos de teste.

Um software sem testes é um software cuja correção você nunca vai poder provar. Testar não é apertar botões até nada quebrar: é um processo de verificação sistemática que responde à pergunta "o software faz o que deveria fazer, nos casos que importam?"

7.1 Os níveis de teste

Teste unitário verifica a menor unidade de código isoladamente. É rápido, barato e deve rodar a cada alteração. **Teste de integração** verifica se as partes funcionam juntas. **Teste de sistema** valida o conjunto completo contra os requisitos. **Teste de aceitação** é feito com o usuário final, muitas vezes por meio de pesquisa de uso.

7.2 O ciclo do defeito

Um defeito tem um ciclo de vida bem conhecido: **reprodução**, **diagnóstico**, **correção**, **validação** e **encerramento**. A etapa de reprodução é a mais importante: um defeito que não pode ser reproduzido de forma consistente não pode ser corrigido com confiança. Por isso a primeira coisa a fazer ao receber um relato de defeito é reconstruir o cenário.

7.3 Testes automatizados em Python

```
import unittest

class TestMedia(unittest.TestCase):

    def test_media_de_lista_normal(self):
        self.assertAlmostEqual(media([8, 9, 7]), 8.0)

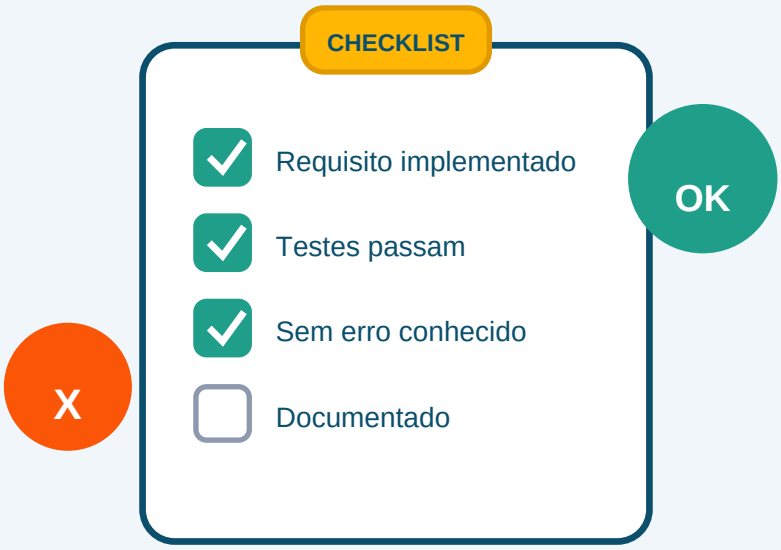
    def test_lista_vazia_falha(self):
        with self.assertRaises(ValueError):
            media([])

    def test_media_rejeita_texto(self):
        with self.assertRaises(TypeError):
            media([8, "9", 7])
```

O padrão **Organizar, Agir, Verificar** deixa cada teste legível e isolado. O nome do teste deve descrever o comportamento esperado, e não a implementação: **test_lista_vazia_falha** é melhor do que **test_2**, porque continua válido mesmo se você reescrever a função.

7.4 A pirâmide de testes

Uma distribuição saudável tem muitos testes unitários (rápidos e baratos), alguns testes de integração e poucos testes de ponta a ponta (lentos e frágeis). Quando a pirâmide está invertida, a suíte demora horas para rodar e falha por motivos que não têm relação com o código alterado.



Testar antes de entregar

Mão na massa - Atividade 4

Escreva a suíte de testes de um sistema de pedidos: validação de preço, de quantidade, de estoque e o cálculo do total com desconto progressivo. Depois introduza de propósito um erro no código e mostre como o teste falha. Essa é a prova de que o teste realmente verifica alguma coisa.

CAPÍTULO 8 - INTELIGÊNCIA ARTIFICIAL

Objetivo: Compreender como a IA aprende, o que ela é capaz de fazer e o que não é, e usar prompts de forma eficaz e responsável.

O termo inteligência artificial é amplo. Em uso corrente, designa sistemas que produzem resultados a partir de exemplos, em vez de seguir regras escritas explicitamente. O *machine learning* é a área que constrói esses sistemas: em vez de programar a regra, você apresenta exemplos e o algoritmo encontra um padrão.

8.1 Como um modelo aprende

De forma simplificada, o treinamento segue quatro etapas. Primeiro, você reúne dados e os rotula. Depois, escolhe um modelo e um algoritmo. Em seguida, o modelo é ajustado repetidamente para reduzir o erro nas previsões, medido por uma função de perda. Por fim, o modelo é avaliado em dados que ele nunca viu, para verificar se generaliza de verdade.

A distinção entre dados de **treino** e dados de **teste** é essencial. Se você avalia o modelo nos mesmos dados usados no treino, o resultado é otimista e enganoso. Vale a mesma lógica para a vida: ninguém se prepara para uma prova resolvendo exatamente o ano que vai cair.

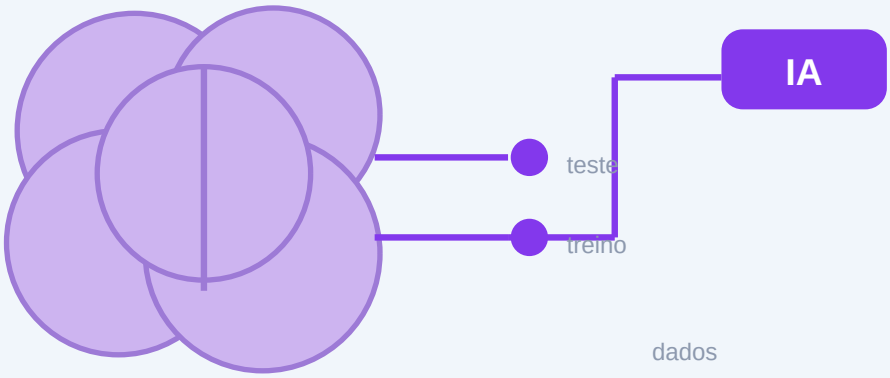
8.2 Uso responsável de IA

A capacidade de gerar texto, código e imagens abriu possibilidades reais e também riscos reais. **Alucinações:** a IA pode afirmar com confiança coisas falsas. **Viés:** modelos aprendem os desvios presentes nos dados. **Impacto ambiental:** treinar modelos grandes consome energia e água em escala. **Privacidade:** dados enviados a serviços externos podem ser retidos ou aproveitados indevidamente.

A postura profissional é simples: use a IA como **assistente que acelera o trabalho**, nunca como **fonte de verdade**. Verifique toda informação, declare o uso e mantenha a responsabilidade humana sobre o resultado final.

8.3 Como escrever um bom prompt

Um prompt eficaz tem quatro partes: **papel** (quem a IA deve ser), **contexto** (o que ela precisa saber), **tarefa** (o que deve fazer, de forma delimitada) e **formato** (como entregar o resultado). Exemplo: "Atue como tutor de Python para adolescentes. Estou resolvendo um exercício de média com notas. Explique por que input() devolve texto e mostre dois exemplos de conversão, no formato de tabela com as colunas entrada e saída."



Como a máquina aprende com exemplos

Desafio

Peça a uma ferramenta de IA que explique um conceito do curso e depois verifique a explicação contra o conteúdo deste livro. Escreva quais afirmações estavam corretas e quais estavam erradas. Essa verificação é a habilidade mais valiosa que você vai construir neste capítulo.

CAPÍTULO 9 - METODOLOGIAS ÁGEIS, CIBERSEGURANÇA E ÉTICA

Objetivo: Trabalhar com Scrum e Kanban, identificar ameaças de segurança e tomar decisões éticas justificáveis.

9.1 Scrum e Kanban

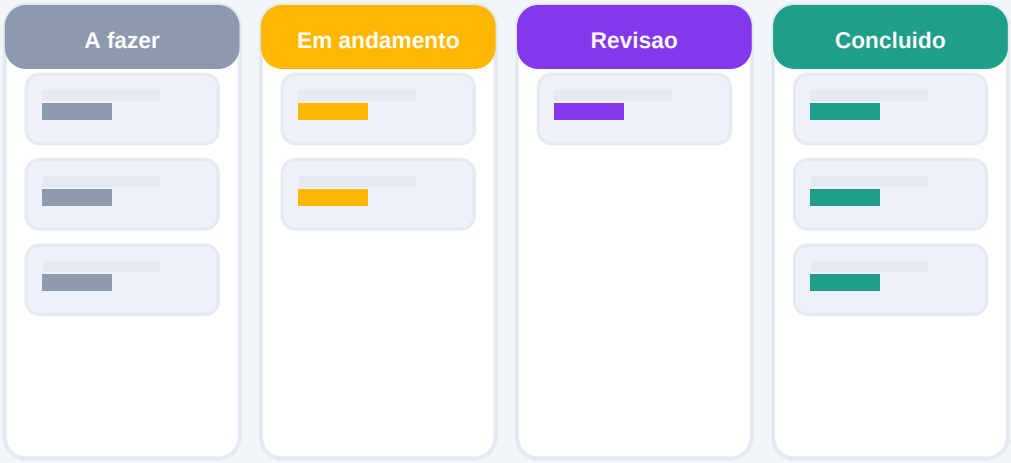
O Scrum organiza o trabalho em ciclos curtos, chamadas **sprints**, nos quais a equipe entrega um incremento funcional. Os papéis são **Product Owner**, que define prioridades e o que tem valor para o usuário; **Scrum Master**, que remove impedimentos e facilita o processo; e a **Equipe de Desenvolvimento**, que constrói. Kanban, por outro lado, organiza o fluxo em um quadro visual com colunas como "a fazer", "em andamento" e "pronto", limitando o trabalho em andamento para reduzir desperdício.

9.2 Os tipos mais comuns de ameaça

Injeção de SQL ocorre quando dados do usuário são concatenados em uma consulta. **XSS** acontece quando o navegador executa código inserido em uma página. **Falha de autenticação** acontece quando credenciais são aceitas sem verificação suficiente. **Exposição de dados sensíveis** acontece quando segredos ficam no código ou no navegador, em vez de ficarem no servidor. A defesa quase sempre passa por **validação de entrada**, **consultas parametrizadas**, **criptografia** e **menor privilégio**.

9.3 Ética: as perguntas certas

Antes de construir, vale responder: quem é afetado? Quem fica de fora? O que acontece com os dados coletados? A solução resolve o problema ou apenas o desloca? De quem é a responsabilidade final? Essas perguntas não substituem a decisão técnica: elas dão contexto para ela.



Quadro Kanban: organizar o fluxo de trabalho



Proteger dados e privacidade

Mão na massa - Atividade 5

Monte um quadro Kanban para a equipe do seu projeto final, com colunas Backlog, A fazer, Em andamento, Revisão e Concluído. Defina um cartão de trabalho com descrição, critério de pronto e responsável. Ao fim da sprint, apresente os cartões movidos e o que travou cada um.

CAPÍTULO 10 - PROJETO FINAL: TECNOLOGIA A SERVIÇO DA COMUNIDADE

Objetivo: Entregar um projeto completo, testado, documentado e publicado.

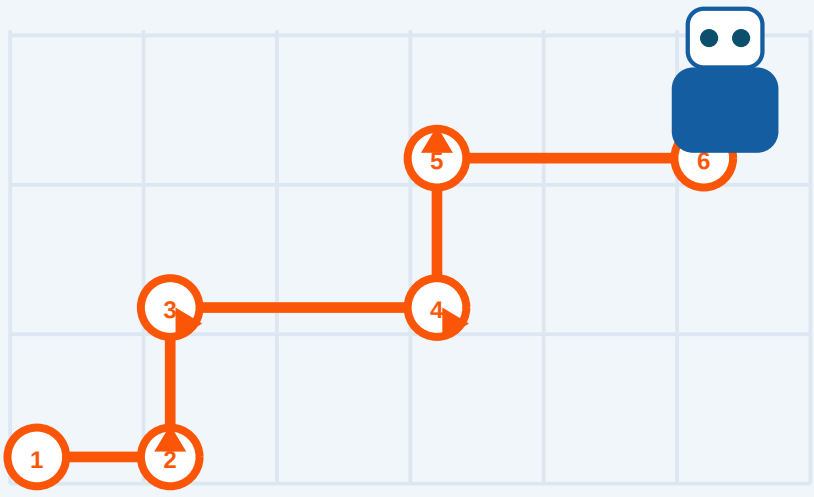
O projeto final reúne tudo o que você aprendeu. A definição de pronto não é "funciona na minha máquina": é um critério objetivo, verificável por terceiros e declarado antes de começar.

10.1 Do problema à entrega

1. Problema. Defina o problema real, quem o sofre e qual é a dor. **2. Escopo.** Defina o que entra e o que fica de fora, por escrito. **3. Requisitos.** Escreva o que o sistema deve fazer, na linguagem do usuário. **4. Arquitetura.** Escolha as tecnologias e justifique cada escolha. **5. Implementação.** Desenvolva em ciclos curtos, com um commit a cada funcionalidade. **6. Testes.** Escreva os testes enquanto desenvolve. **7. Documentação.** README, decisões de projeto e instruções de uso. **8. Publicação.** Publique e anuncie.

10.2 Critérios de qualidade do projeto

- Funciona conforme os requisitos escritos
- Não contém erro conhecido no caminho principal
- Possui testes automatizados que passam
- O README explica como executar o projeto
- A interface é acessível e testada com teclado
- Não expõe dados pessoais nem segredos
- Está publicado em repositório público, com licença



Do problema à entrega: um passo de cada vez

Desafio final

Apresente seu projeto em cinco minutos: o problema, a solução, uma demonstração ao vivo, o que você aprendeu e o que ficou para trás. Depois responda a uma pergunta da turma. Um

bom projeto de software se explica sozinho em cinco minutos.

CAPÍTULO 11 - INGLÊS TÉCNICO PARA PROGRAMADORES

> **Objetivo:** Ler documentação, mensagens de erro e discussões técnicas em inglês.

Toda a documentação técnica de referência está em inglês. A mensagem de erro que aparece no terminal, o nome de uma função, o título de um tutorial, as discussões em um repositório: tudo isso está em inglês. A boa notícia é que o vocabulário técnico é finito e se repete. Dominar cerca de duzentos termos cobre a maior parte do que você vai ler.

11.1 Mensagens de erro em inglês

A mensagem de erro é, na verdade, uma descrição técnica. **TypeError** significa que o tipo do valor não era o esperado. **ValueError** significa que o valor é do tipo certo, mas inválido. **NameError** significa que o nome não existe no escopo. **KeyError** significa que a chave não existe no dicionário. **IndexError** significa que o índice estava fora do intervalo. **AttributeError** significa que o objeto não tem aquele atributo. Traduzir a estrutura da mensagem é o passo mais rápido para depurar.

11.2 Verbos técnicos

Os verbos da documentação indicam o que você pode fazer: **get** obtém, **set** define, **create** cria, **update** atualiza, **delete** remove, **parse** interpreta, **validate** valida, **return** devolve, **raise** lança um erro, **handle** trata, **deploy** publica em produção, **refactor** reescreve sem mudar o comportamento.

11.3 Frases do dia a dia

Em uma revisão de código: "Could you please refactor this function?" (não "change it"). Em um relato de defeito: "Steps to reproduce" (os passos para reproduzir). Em um pull request: "This PR adds validation to the login form." (adiciona validação). Em uma reunião: "What is the definition of done?" (quando está pronto). Em uma revisão: "Looks good to me" (está bom para mim). Leia essas frases como vocabulário ativo, e você já consegue participar de uma equipe internacional.



Ler documentação em inglês faz parte do trabalho

Desafio

Escreva o README do seu projeto final em inglês, com as seções Object, Features, Tech Stack, How to Run, Tests, License e Credits. Peça a um verificador de gramática em inglês para revisar. Use também o glossário de termos em inglês e português como referência.

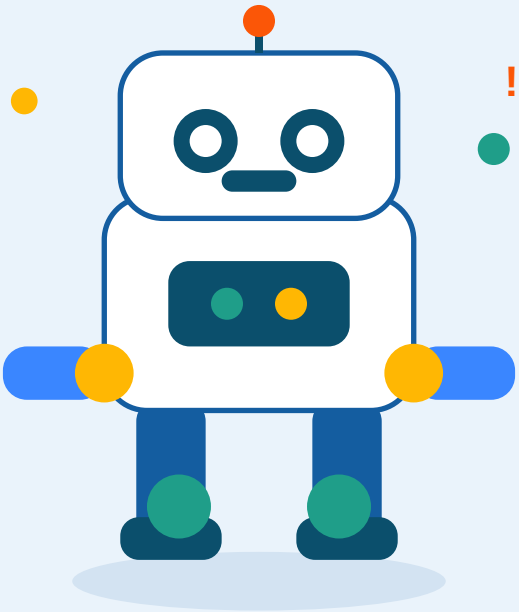
CAPÍTULO 12 - RECAPITULANDO E CONTINUANDO

O que você domina agora:

• Pensar de forma algorítmica e aplicar os quatro pilares
• Escrever e analisar algoritmos, com atenção à complexidade
• Programar em Python, com tratamento de erros e testes
• Construir interfaces web semânticas e responsivas
• Modelar dados e escrever consultas SQL eficientes
• Versionar projetos e colaborar com Git e GitHub
• Escrever e automatizar testes de software
• Compreender o funcionamento e os limites da IA
• Trabalhar com Scrum e Kanban, pensando em segurança e ética
• Ler documentação técnica em inglês

Onde continuar:

- **Documentação oficial do Python:** docs.python.org/pt-br/3/
- **SQLite:** sqlite.org/docs.html e sqlitebrowser.org
- **Pro Git (livro gratuito):** git-scm.com/book/pt-br
- **CS Unplugged:** classic.csunplugged.org
- **Próximo tema, à sua escolha:** visão computacional, redes, devops ou ciência de dados



Você já faz parte dessa história

Parabéns. Você não apenas aprendeu a usar tecnologia: aprendeu a criar, testar, documentar e publicar. Esse é o perfil de quem constrói ferramentas que melhoram a vida de outras pessoas.

PLANNER DO PROJETO FINAL

Preencha antes de começar a codar. Um projeto bem planejado economiza semanas de retrabalho.

Nome do projeto

Problema que eu resolvo

Para quem é

Escopo: o que ENTRA no projeto

Escopo: o que NÃO entra (por enquanto)

Regras de negócio principais

Tecnologias que vou usar e por quê

Casos de uso: o que o usuário faz

Como vou testar

O que já está pronto e o que falta

Riscos e como vou lidar com eles

LINKS RÁPIDOS DE FERRAMENTAS

Clique no link para abrir a ferramenta no navegador. Todas são gratuitas para uso educacional.

FERRAMENTA	O QUE SERVE	LINK
Python no navegador	Programar sem instalar nada	**colab.research.google.com**
Documentação do Python	Referência oficial das funções	**docs.python.org/pt-br/3**
SQLite Browser	Criar e consultar bancos de dados	**sqlitebrowser.org**
Visualg	Algoritmos com fluxogramas visuais	**visualg3.com.br**
GitHub	Hospedar código e colaborar	**github.com**
GitHub Pages	Publicar seu site de graça	**pages.github.com**
Pro Git (livro livre)	Git do zero ao profissional	**git-scm.com/book/pt-br**
CS Unplugged	Conceitos de computação sem computador	**classic.csunplugged.org**
Laboratório Tech Social	Exercícios interativos e material do professor	**marquezbertin.github.io/laboratorio-tech-social**
QA_RUN	Questionários e quizzes de tecnologia	**marquezbertin.github.io/QA_RUN**
Material completo	Planos de aula e repositório do projeto	**github.com/Marquezbertin/Material-Didatico-Tecnologia**

Referência de estudo: a metodologia deste livro segue a abordagem de *Computer Science Unplugged* (Brian Harwell e colaboradores), na tradução brasileira da Fundação de Apoio à Educação e Cultura, e as documentações oficiais de Python, SQLite, Git e GitHub.

GLOSSÁRIO

TERMO	SIGNIFICADO
Algoritmo	Sequência precisa de passos que resolve um problema
API	Interface programática que permite a comunicação entre sistemas
Big-O	Notação que descreve o crescimento do custo de um algoritmo
Branch	Linha independente de desenvolvimento dentro de um repositório
Bug	Defeito em um programa que viola um requisito esperado
Commit	Ponto registrado no histórico do Git, com autor e mensagem
CSS	Linguagem que define a apresentação de páginas web
Dependência externa	Biblioteca de terceiros usada por um projeto
Documentação executável	Documentação que contém testes e é verificada na execução
Framework	Estrutura que oferece padrões e ferramentas prontas
Git	Sistema de controle de versão de arquivos
GitHub	Plataforma web de hospedagem de repositórios Git
HTML	Linguagem de marcação que define a estrutura do conteúdo web
Índice	Estrutura auxiliar que acelera consultas ao banco de dados
JavaScript	Linguagem que define o comportamento dinâmico das páginas web
Join	Operação SQL que combina linhas de duas ou mais tabelas
Kanban	Método visual de organização do fluxo de trabalho
Modelagem de dados	Processo de estruturar dados em entidades e relacionamentos
Pull request	Proposta de integração de uma branch, sujeita a revisão
Python	Linguagem de programação de alto nível, legível e versátil
Query	Consulta enviada ao banco de dados para obter ou alterar dados
Recursão	Técnica em que uma função chama a si mesma com caso menor
Scrum	Estrutura ágil com sprints incrementais, papéis e eventos definidos
SQL	Linguagem para consultar e manipular bancos de dados relacionais
Teste unitário	Verificação automatizada de uma unidade isolada de código
Versionamento	Prática de registrar e controlar mudanças em um projeto